

SERIAL PERIPHERAL INTERFACE
(SPI PROTOCOL)
IMPLEMENTATION
USING VERILOG

PREPARED BY:

Nensi Thummar

Contents

1. ABSTRACT	3
2. OBJECTIVE.....	3
3. INTRODUCTION	4
MODES OF SPI	5
4. MULTI-SLAVE CONFIGURATIONS.....	6
1. Multi-slave	6
2. Daisy Chain.....	6
5. SYSTEM ARCHITECTURE	7
6. SPI MASTER DESIGN	8
7. SPI MASTER WORKING PRINCIPLE	8
8. SPI SLAVE DESIGN	8
9. SPI SLAVE WORKING PRINCIPLE.....	8
10. FINITE STATE MACHINE (FSM) OF SPI.....	9
FSM States Description	9
1. IDLE State	9
2. TRANSFER State.....	9
3. DONE State.....	10
FSM State Transition Table	10
FSM State Diagram.....	11
11. TESTBENCH DESIGN	11
Testbench Features	11
Test Cases.....	11
12. SIMULATION RESULTS	12
13. APPLICATIONS:	13
14. CHALLENGES FACED.....	14
15. LEARNING OUTCOMES	14
16. ADVANTAGES OF SPI.....	14
17. DISADVANTAGES OF SPI	15
18. FUTURE ENHANCEMENTS	15
19. CONCLUSION	15
20. REFERENCES.....	15

1. ABSTRACT

This project presents the design and implementation of a configurable Serial Peripheral Interface (SPI) protocol using Verilog HDL. The design includes one SPI master and multiple SPI slave devices supporting full-duplex synchronous serial communication. The implementation supports configurable clock polarity (CPOL), clock phase (CPHA), programmable clock division, and parameterized data width.

The SPI master controls data transmission through MOSI, receives data through MISO, and selects slave devices using dedicated chip-select lines. Three SPI slave modules are integrated to demonstrate multi-slave communication capability. A detailed testbench is developed to verify functionality under different transfer conditions. Simulation results confirm successful data exchange between master and slave devices with proper timing synchronization.

The project demonstrates concepts of serial communication protocols, finite state machine design, synchronous digital systems, and parameterized hardware modelling using Verilog HDL.

2. OBJECTIVE

- To design an SPI master using Verilog HDL
- To implement SPI slave devices supporting configurable SPI modes
- To support full-duplex serial communication
- To implement multi-slave communication using chip-select lines
- To verify SPI functionality through simulation and waveform analysis
- To understand synchronous communication protocols and FSM-based control logic

3. INTRODUCTION

Serial Peripheral Interface (SPI) is a high-speed synchronous serial communication protocol widely used for short-distance communication between microcontrollers, processors, sensors, memory devices, and peripheral components in embedded systems. SPI was originally developed by Motorola and has become one of the most commonly used communication protocols in digital electronics due to its simplicity, speed, and full-duplex communication capability.

SPI follows a master-slave architecture in which one device acts as the master and controls the communication process, while slave devices respond to the master. The master generates the clock signal and selects the target slave device for communication. Data transfer takes place synchronously with respect to the generated clock signal.

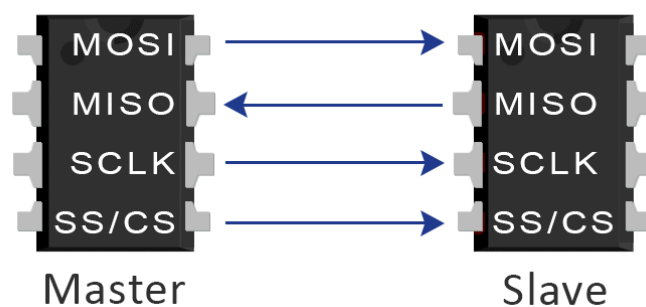


Fig. 1: SPI Configuration

The SPI protocol uses four signals:

1. **SCLK (Serial Clock)**
The clock signal generated by the master device used to synchronize data transmission between master and slave.
2. **MOSI (Master Out Slave In)**
The data line used to transfer serial data from the master to the slave.
3. **MISO (Master In Slave Out)**
The data line used to transfer serial data from the slave to the master.
4. **CS/SS (Chip Select/Slave Select)**
An active-low control signal used by the master to select a specific slave device for communication.

SPI supports **full-duplex communication**, meaning data transmission and reception occur simultaneously. While the master transmits data through MOSI, it can also receive data from the slave through MISO during the same clock cycle. This feature makes SPI faster and more efficient than many other serial communication protocols.

One of the major advantages of SPI is its high-speed operation because it does not require complex addressing or acknowledgment mechanisms.

MODES OF SPI

SPI communication operates using configurable clock settings known as **Clock Polarity (CPOL)** and **Clock Phase (CPHA)**. These settings define the clock idle state and determine the edge of the clock at which data is sampled and shifted. Based on CPOL and CPHA values, SPI supports four communication modes:

SPI MODE	CPOL	CPHA	CLOCK POLARITY IN IDLE STATE	CLOCK PHASE USED TO SAMPLE AND SHIFT DATA
0	0	0	Logic low	Data sampled on the rising edge and shifted out on the falling edge
1	0	1	Logic low	Data sampled on the falling edge and shifted out on the rising edge
2	1	0	Logic high	Data sampled on the falling edge and shifted out on the rising edge
3	1	1	Logic high	Data sampled on the rising edge and shifted out on the falling edge

Below is a diagram of the Clock Polarities and Clock Phases of SPI Modes:

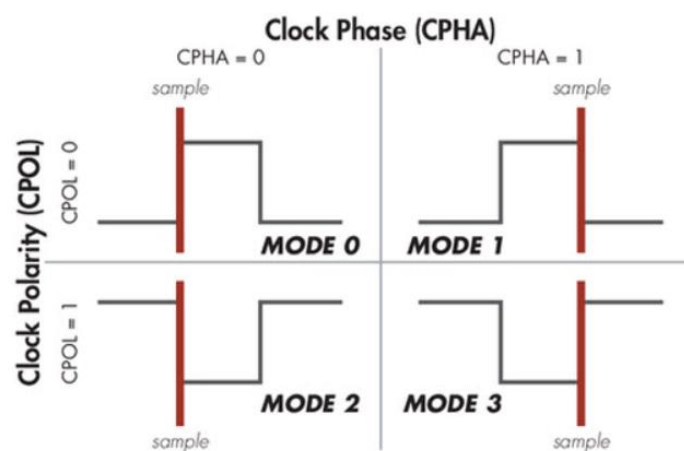


Fig. 2: 4 Modes of SPI

4. MULTI-SLAVE CONFIGURATIONS

There are two common multi-slave SPI configurations:

1. Multi-slave
2. Daisy Chain

1. Multi-slave

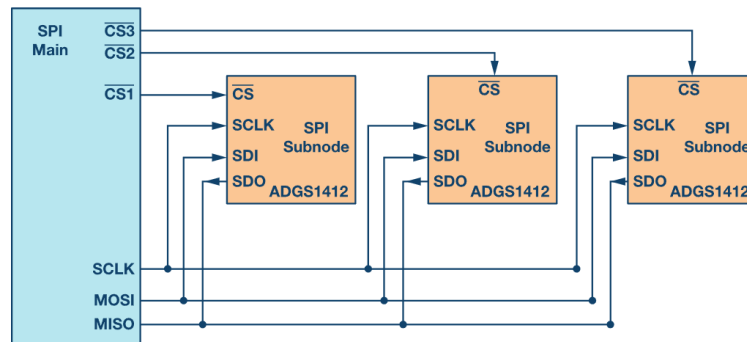


Fig. 3: Multi-slave configuration

2. Daisy Chain

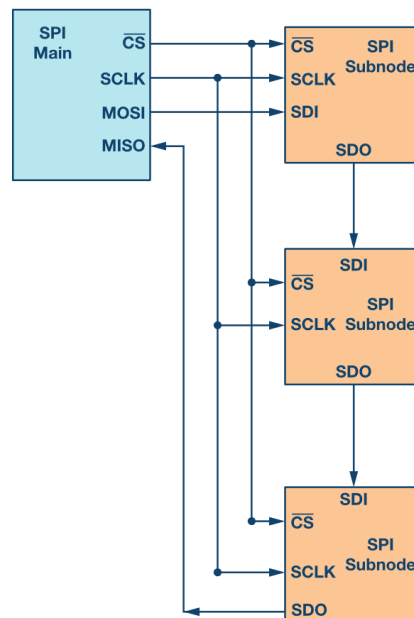


Fig. 4: Daisy Chain configuration

5. SYSTEM ARCHITECTURE

The implemented SPI system consists of one SPI master and three SPI slave devices. The master communicates with the selected slave through shared SCLK, MOSI, and MISO lines, while individual chip-select signals are used to select the desired slave. The architecture supports full-duplex communication and demonstrates multi-slave SPI operation.

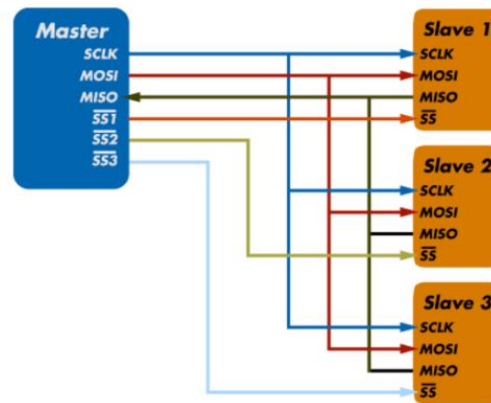
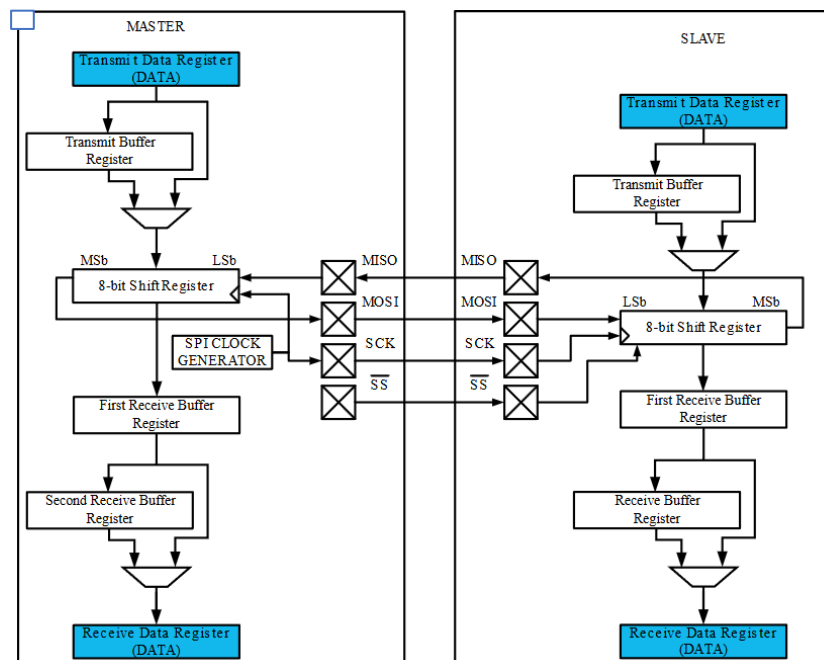


Fig. 5: Our system architecture

The SPI master generates the serial clock and manages communication through chip-select signals. The selected slave exchanges data with the master using the MOSI and MISO lines. All slave devices share the communication bus, while only the selected slave actively participates in the data transfer.



Note: The implemented design uses a 16-bit shift register.

Fig. 6: SPI Master-Slave Internal Functional Block Diagram

6. SPI MASTER DESIGN

Features of SPI Master:

- Parameterized data width
- Programmable clock divider
- Configurable CPOL and CPHA
- FSM-based control
- Multi-slave support
- Full-duplex transmission

7. SPI MASTER WORKING PRINCIPLE

The SPI master module controls the complete communication process. Initially, the controller remains in the IDLE state waiting for the start signal. Once the start signal is asserted, the selected slave device is activated through the chip-select line and transmission begins.

The master generates the serial clock (SCLK) using a programmable clock divider. Depending on CPOL and CPHA configuration, data sampling and shifting operations are performed on appropriate clock edges.

Transmission data is loaded into a shift register. During every transfer cycle:

- Data is transmitted serially through MOSI
- Data is simultaneously received through MISO
- The receive data is stored in the receive register

A counter tracks the number of transferred bits. After completion of all bits, the controller enters the DONE state, deactivates the slave select line, and returns to the IDLE state.

8. SPI SLAVE DESIGN

Features of SPI Slave:

- Parameterized data width
- Supports CPOL and CPHA configurations
- Shift-register based transmission and reception
- Tri-state MISO output
- Full-duplex communication
- Compatible with multi-slave architecture

9. SPI SLAVE WORKING PRINCIPLE

The SPI slave module communicates with the SPI master whenever its chip-select (CS) signal is asserted. When the slave is selected, the transmit data is loaded into an internal shift register and the slave becomes ready for data transfer.

The slave receives serial data from the MOSI line and simultaneously transmits data through the MISO line. Depending on the selected CPOL and CPHA configuration, data sampling and shifting operations occur on the appropriate clock edges.

During each clock cycle:

- Data received through MOSI is shifted into the receive register.
- Data stored in the transmit register is shifted out through MISO.

After all bits are transferred, the received data is stored in the receive register and the slave waits for the next communication request from the master.

10. FINITE STATE MACHINE (FSM) OF SPI

The SPI master controller is designed using a Finite State Machine (FSM) to manage the complete communication process. The FSM controls slave selection, clock generation, serial data transfer, and transaction completion. The design consists of three states:

1. IDLE State
2. TRANSFER State
3. DONE State

The FSM ensures synchronized and organized SPI communication between the master and slave devices.

FSM States Description

1. IDLE State

This is the waiting state of the SPI master.

In this state:

- SPI clock stays at idle value
- No slave is selected
- busy = 0
- done = 0
- Master waits for start signal

When start = 1:

- The slave is selected
- Data is loaded into the shift register
- Counter is initialized
- busy becomes 1
- FSM moves to TRANSFER state

2. TRANSFER State

In this state, actual SPI communication takes place.

In this state:

- SPI clock is generated
- Data is sent through MOSI

- Data is received through MISO
- Data shifting and sampling are performed
- Bit counter decreases after each transfer

Transfer process:

- On one clock edge:
 - Data is shifted out on MOSI
- On the other clock edge:
 - Data is sampled from MISO

After all bits are transferred:

- busy = 0
- done = 1
- FSM moves to DONE state

3. DONE State

This state ends the SPI communication.

In this state:

- Slave select lines are disabled
- SPI clock returns to idle state
- MOSI is cleared
- Transfer is completed

Next State:

- FSM returns to IDLE state

FSM State Transition Table

Present State	Condition	Next State	Operation
IDLE	start = 0	IDLE	Wait for communication
IDLE	start = 1	TRANSFER	Initialize SPI transfer
TRANSFER	Transfer incomplete	TRANSFER	Continue serial communication
TRANSFER	All bits transferred	DONE	Finish transaction
DONE	Automatic	IDLE	Return to idle condition

FSM State Diagram

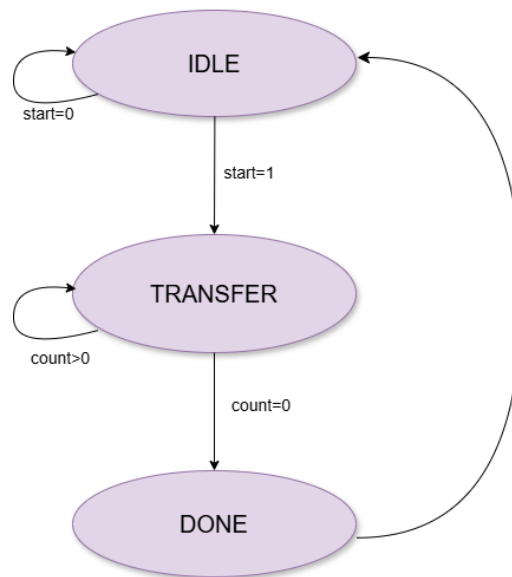


Fig. 7: Complete FSM

11. TESTBENCH DESIGN

Testbench Features

- Generates system clock
- Applies reset
- Sends multiple transactions
- Tests communication with 3 slaves
- Monitors signals using \$monitor
- Dumps waveform using \$dumpvars

Test Cases

Test Case	Master TX	Slave Selected	Expected RX
TC1	ABCD	Slave0	1234
TC2	1234	Slave1	5555
TC3	6789	Slave2	F0F0

12. SIMULATION RESULTS



Fig. 8: Complete simulation with all three transfers

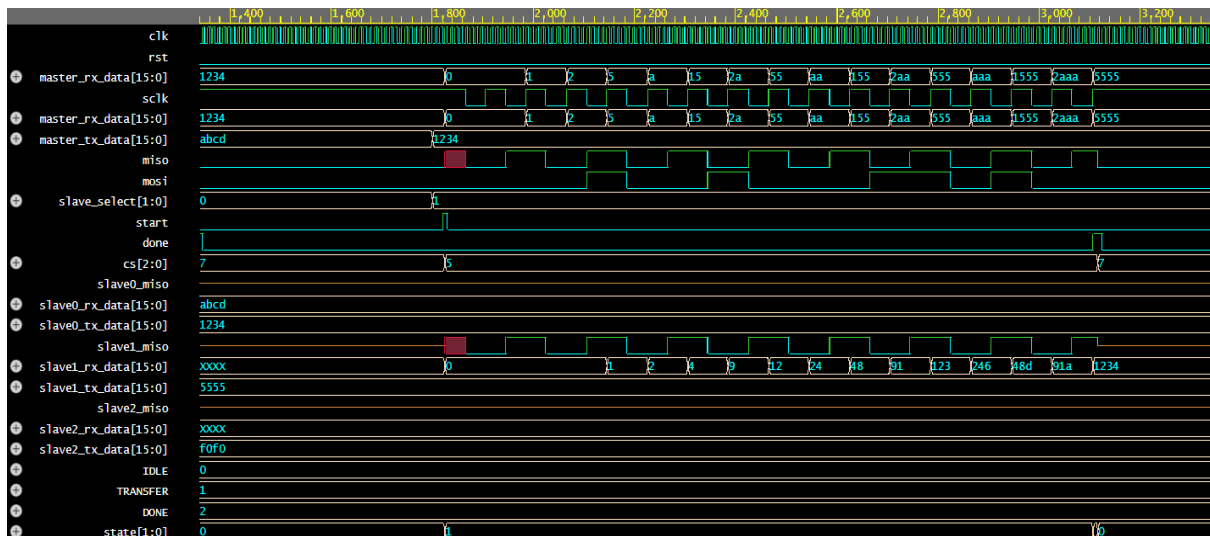


Fig. 9: 1st Transfer

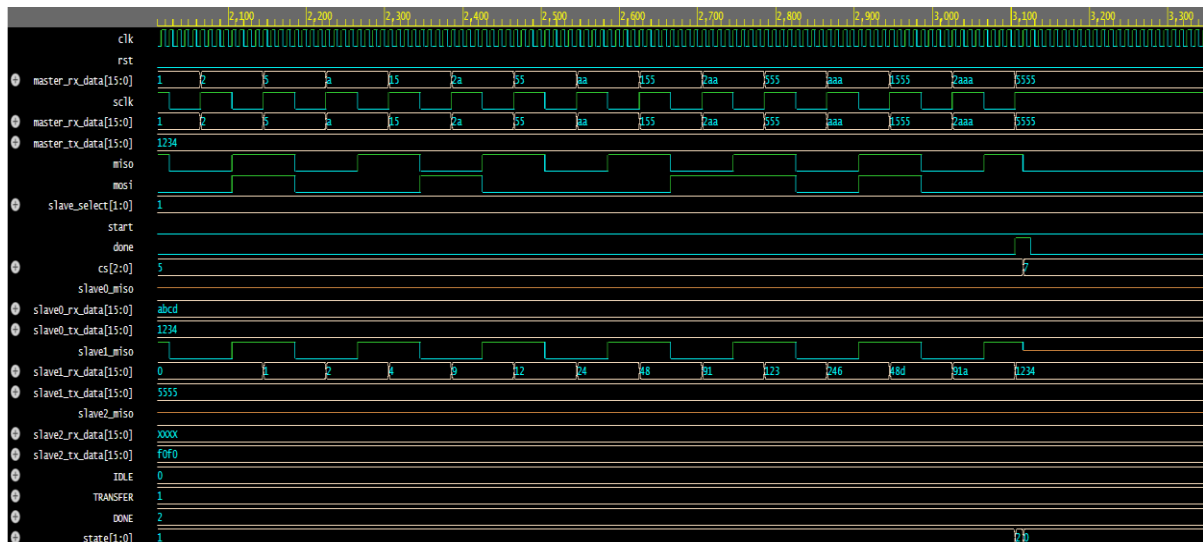


Fig. 10: 2nd Transfer

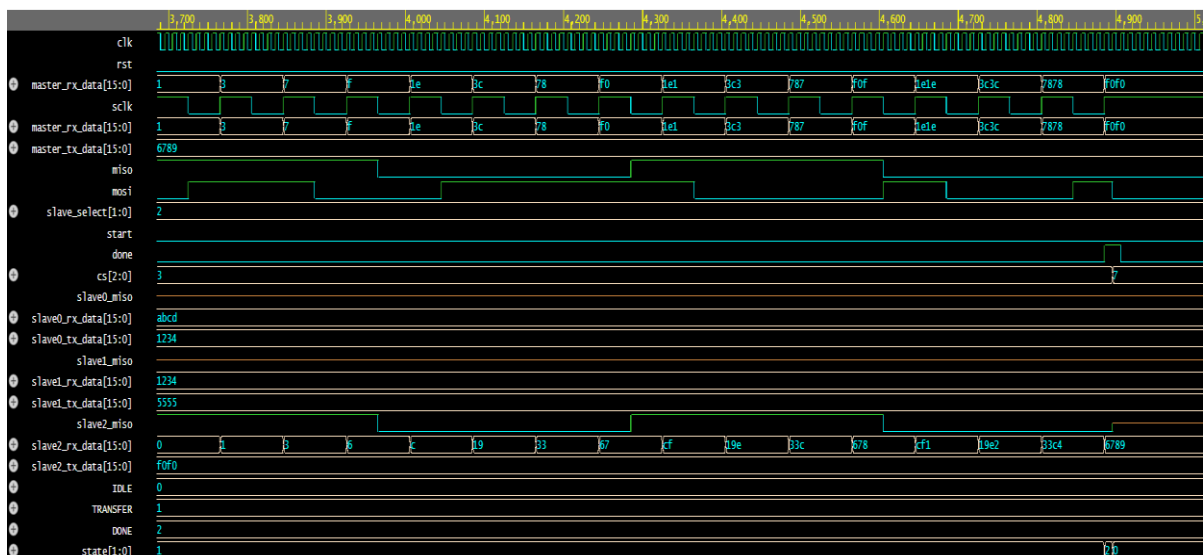


Fig. 11: 3rd Transfer

13. APPLICATIONS:

SPI is commonly used in applications requiring fast data exchange such as:

- Flash memory communication
- Sensor interfacing
- ADC and DAC communication
- Display modules
- FPGA and microcontroller systems
- Wireless communication modules

14. CHALLENGES FACED

During the implementation of the SPI protocol, the following challenges were encountered:

1. SPI Mode Implementation

Handling different CPOL and CPHA combinations to ensure correct data sampling and shifting.

2. Clock Synchronization

Maintaining accurate timing between SCLK, MOSI, and MISO signals during data transfer.

3. Multi-Slave Communication

Managing slave selection and ensuring that only the selected slave responds during communication.

4. FSM Debugging

Verifying correct state transitions and transaction completion through simulation waveforms.

5. Functional Verification

Developing test cases to validate successful communication between the master and multiple slave devices.

15. LEARNING OUTCOMES

Through this project, the following concepts were learned:

1. SPI Protocol Understanding

Gained knowledge of SPI communication, signals, and multi-slave architecture.

2. FSM Implementation

Understood the use of Finite State Machines for controlling communication processes.

3. Serial Data Communication

Learned how data is transmitted and received using shift registers in SPI communication.

4. Simulation and Verification

Developed skills in writing testbenches and analyzing waveforms to verify functionality.

5. Digital System Integration

Gained experience in integrating master and slave modules to build a complete communication system.

16. ADVANTAGES OF SPI

- High-speed communication
- Supports full-duplex data transfer
- Simple hardware implementation
- Faster than I2C protocol
- Supports multiple slave devices

17. DISADVANTAGES OF SPI

- Requires more wires
- A separate chip select is needed for each slave
- No built-in error checking
- Not suitable for long-distance communication

18. FUTURE ENHANCEMENTS

- FIFO buffering
- Interrupt support
- DMA integration
- CRC/error checking
- Dynamic slave selection support
- SPI burst transfer support

19. CONCLUSION

The SPI communication protocol was successfully implemented using Verilog HDL with configurable master and slave modules. The design supports full-duplex synchronous communication, multiple SPI modes, programmable clock division, and multi-slave interfacing. Simulation results verified the correct transmission and reception of serial data between the master and slave devices.

20. REFERENCES

1. Texas Instruments – SPI Communication Basics (Application Report)
2. Analog Devices-SPI Protocol Explanation
3. GeeksforGeeks – Serial Peripheral Interface (SPI) Protocol
4. Microchip Technology-SPI Online Documentation
5. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, Pearson Education.
6. M. Morris Mano and Michael D. Ciletti, *Digital Design*, Pearson Education.